

Server-Client Communication Application Plan

Qt 5.14.0 desktop client, Qt server, MySQL database access through server only, with TCP load balancing.

CLIENT

Qt Widgets application with database tree, table browser, editable row grid, and live connection state.

SERVER

Qt QCoreApplication service that owns all MySQL access, validates requests, and executes SQL through pooled connections.

SCALING

Multiple server instances behind HAProxy or Nginx stream load balancing.

DATABASE

MySQL-compatible backend. Exact target version should be confirmed before final driver packaging.

1. Main Goal

Build a server-client database management application. The client connects to the server, shows databases in a left sidebar, expands a selected database into tables, shows the selected database tables in the right panel, and allows users to view, insert, save, and delete table rows. The client must never connect to MySQL directly.

2. High-Level Architecture

```
Client App 1 ----\  
Client App 2 -----+--> Load Balancer --> Qt Server Instance 1 --> MySQL  
Client App N ----/      --> Qt Server Instance 2 --> MySQL  
                        --> Qt Server Instance N --> MySQL
```

The load balancer owns the public TCP endpoint. Each server instance is mostly stateless and can serve any client. The database remains protected behind the server layer.

3. Project Structure

Project	Type	Responsibility
Common	Shared library	Protocol framing, request and response types, error codes, constants, serialization helpers.
Server	Qt console/service app	TCP accept loop, session management, authentication, request routing, MySQL connection pooling, validation, SQL execution.
Client	Qt Widgets app	Connection management, database tree, table list view, editable table grid, add/save/delete actions, user feedback.
Deploy	Configuration	HAProxy or Nginx stream config, server instance config, startup scripts, environment examples.

4. Load Balancing Requirement

Use a proven TCP load balancer instead of building a custom load balancer in Qt. Recommended choices are HAProxy or Nginx stream. The client connects to the load balancer address, not to a specific server instance.

- Use health checks to remove failed server instances from rotation.
- Use TCP mode because the application protocol runs over Qt sockets.
- Use client reconnect logic so a disconnected client can reconnect through the load balancer.
- Keep server session state minimal so reconnection to another server is possible.

Example HAProxy Layout

```
frontend qt_db_client
    bind *:7000
    mode tcp
    default_backend qt_db_servers

backend qt_db_servers
    mode tcp
    balance leastconn
    option tcp-check
    server server1 10.0.0.11:7100 check
    server server2 10.0.0.12:7100 check
    server server3 10.0.0.13:7100 check
```

5. Communication Protocol

Use framed binary or compact JSON messages over TCP. Each message should include a request ID so responses can be matched to requests even when multiple operations are active.

Request	Purpose
HELLO / AUTH	Open a client session and authenticate if required.
HEARTBEAT	Check server availability and update connection state in the client.
LIST_DATABASES	Return visible database names.
LIST_TABLES	Return tables for one database.
GET_TABLE_SCHEMA	Return columns, data types, nullable flags, default values, and primary key info.
GET_TABLE_ROWS	Return paged table rows.
INSERT_ROW	Insert a new row after client and server validation.
UPDATE_ROW	Save changes for an existing row.
DELETE_ROW	Delete a row using primary key values.

6. Server Design

- Use `QTcpServer` for incoming client connections.
- Use event-driven socket handling for large numbers of idle or low-activity clients.
- Use a worker thread pool for database tasks.
- Use a bounded MySQL connection pool. Do not create one database connection per client.
- Validate database names, table names, column names, data types, nullable fields, and primary keys on the server.
- Use parameterized SQL for row insert, update, and delete operations.
- Add queue limits, request timeouts, connection cleanup, and structured error responses.

Serving 100,000 simultaneous users requires multiple server instances, careful OS socket tuning, enough MySQL capacity, load testing, and realistic pagination. The server should be designed for this target, then measured and scaled horizontally.

7. Client Design

- Use a Qt Widgets `QMainWindow` with a split layout.
- Left sidebar: `QTreeView` showing databases and expanded tables.
- Right panel: table list after selecting a database, editable row grid after selecting a table.
- Status area: show Connecting, Connected, Reconnecting, Disconnected, and Error states.
- Use heartbeat and automatic reconnect logic.
- Disable database actions when disconnected or while required schema metadata is unavailable.

8. User Workflow

1. User starts the client application.
2. Client connects to the load balancer endpoint.
3. Server returns the visible database list.
4. Client shows databases in the left sidebar.
5. User double-clicks a database.
6. Client requests tables for that database, expands the tree node, and shows the table list in the right panel.
7. User double-clicks a table.
8. Client requests schema and the first page of rows, then shows content in an editable table view.
9. User clicks Add Row to create a blank row.
10. User fills required not-null fields and clicks Save.
11. Client sends the row to the server. Server validates and inserts it into MySQL.
12. User selects a row and clicks Delete Row. Server deletes it using primary key values.

9. Table Editing Rules

- Load schema before editing so the client knows required fields and primary keys.
- Track new, modified, and deleted rows in the client model.
- Validate not-null fields in the client before Save.
- Repeat validation on the server because the client cannot be trusted as the only validation layer.
- Use primary keys for update and delete. If a table has no primary key, make editing read-only or require a safe row identity strategy.

10. Performance Plan

- Use pagination for table content. Never load a whole large table by default.
- Use server-side metadata caching for database, table, and schema data.
- Use request batching where useful, for example schema plus first page of rows.
- Use async client requests so the UI remains responsive.
- Add server metrics for active clients, request rate, queue depth, DB pool usage, errors, and response time.
- Run load tests with simulated clients before final capacity claims.

11. Security Plan

- Keep all database credentials on the server side.
- Use TLS or a protected private network between client and load balancer when credentials or sensitive data are involved.
- Add user authentication and authorization before exposing real databases.
- Restrict visible databases and tables per user if required.
- Audit insert, update, and delete operations.

12. Implementation Order

1. Create Qt project structure: `Common`, `Server`, `Client`, and `Deploy`.
2. Implement shared protocol framing and message serialization.
3. Implement basic Qt TCP server and client connection.
4. Add connection state display and heartbeat.
5. Add HAProxy or Nginx stream load balancer config.
6. Add `LIST_DATABASES`.
7. Add `LIST_TABLES`.
8. Add schema loading and paged table row loading.
9. Add editable table model with Add Row, Save, and Delete Row.
10. Add server-side validation and parameterized SQL execution.
11. Add reconnect and failover testing through the load balancer.
12. Add load tests and tune server, database pool, OS socket limits, and load balancer settings.

13. Key Risks

Risk	Mitigation
100,000 concurrent users overload one server	Use multiple server instances and TCP load balancing. Keep sockets event-driven and database work pooled.
MySQL becomes the bottleneck	Use pagination, indexes, limited result sizes, metadata caching, and measured connection pool sizes.
Editing unsafe tables without primary keys	Detect primary keys during schema load and make unsafe tables read-only unless a safe identity rule is configured.
Client loses connection during save	Use request IDs, timeouts, reconnect logic, and clear success/failure responses from the server.

14. Deliverables

- Qt 5.14.0 server project.
- Qt 5.14.0 client project.
- Shared protocol library.
- Load balancer configuration.
- MySQL database access layer with connection pooling.
- Client UI for database list, table list, row viewing, adding, saving, deleting, and connection state.
- Load testing tools or scripts for capacity verification.

Final MySQL target should be confirmed before packaging. Common targets are MySQL 5.7, MySQL 8.x, or MySQL Cluster 7.x. Qt SQL driver compatibility and deployment details depend on the exact version and client library.